

Denoising Diffusion Probabilistic Model

Experimental Report

*Detailed documentation of implementation steps, experimental results, and
conclusions for DDPMs.*

June 1, 2026

Report Structure

- **Part I — Background:** Theory of DDPMs and model architecture
- **Part II — Experimental Setup:** Fixed hyperparameters and dataset
- **Part III — Experiments:** Per-experiment results, analysis, and figures (losses, samples, reconstructions, latent spaces)
- **Part IV — Conclusions:** Summary of findings and key differences

Contents

1	Introduction	2
1.1	Purpose of the Comparison	2
2	Background: Denoising Diffusion Probabilistic Models (DDPMs)	2
2.1	Forward diffusion process	2
2.2	Reverse generative process	2
2.3	Training objective	3
2.4	Sampling procedure	3
2.5	How to choose our architecture?	3
2.5.1	Inductive Biases of U-Net Relevant to Reverse Diffusion	4
2.6	Conditioning	6
2.7	How does Conditioning affect?	6
3	Classifier Free Guidance CFG	6
3.1	Background: What Problem Are We Solving?	6
3.2	Part 1: Classifier Guidance (CG)	7
3.3	Part 2: Classifier-Free Guidance (CFG)	9
3.4	Part 3: Direct Comparison	11
4	DDPMs in Contrast to Variational Autoencoders	11
5	Experimental Setup	13
5.1	Dataset	13
5.2	Evaluation Metrics	13
6	Experiment 1	13
6.1	Motivation	13
6.2	Hyperparameters	13
6.3	Evaluation Results	14
6.4	Qualitative Results	14
6.4.1	Random Samples	14
6.5	Analysis and Observations	14
7	Conclusions	15

1 Introduction

This report documents a theoretical and experimental study of **Denoising Diffusion Probabilistic Model (DDPMs)** applied to image generation.

The document is the second part of **comparative analysis between VAEs and Denoising Diffusion Probabilistic Models (DDPMs)**, enabling an apples-to-apples comparison of two dominant paradigms in deep generative modelling.

1.1 Purpose of the Comparison

VAEs and DDPMs represent fundamentally different inductive biases:

- **VAEs** learn an explicit, amortised posterior $q_\phi(\mathbf{z}|\mathbf{x})$ and optimise a tractable Evidence Lower Bound (ELBO), trading reconstruction sharpness for a structured, continuous latent space.
- **DDPMs** learn to reverse a fixed Markov diffusion process, achieving high sample quality at the cost of slow, iterative inference.

Both model families are trained on the **similar architectural backbone with differences we will elaborate later**, the same dataset, so that differences in metrics reflect the generative paradigm rather than implementation disparity.

Note: I tried as much as possible to keep the experiments consistent, but I could not fully achieve that because it requires much more time and additional training.

2 Background: Denoising Diffusion Probabilistic Models (DDPMs)

Denoising Diffusion Probabilistic Models (DDPMs) are a class of likelihood-based generative models that define data generation as the reversal of a gradual noising process. The central idea is to transform complex data distributions into simple noise through a fixed forward process, and then learn a model that reverses this corruption step by step.

2.1 Forward diffusion process

In the forward process, a data sample x_0 is progressively perturbed over a sequence of timesteps $t = 1, \dots, T$ by adding small amounts of Gaussian noise. This defines a Markov chain:

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right),$$

where β_t is a predefined variance schedule controlling the noise level at each step. After sufficiently many steps, the distribution of x_T converges to an isotropic Gaussian, effectively destroying the structure of the original data.

2.2 Reverse generative process

The generative model learns the reverse Markov process:

$$p_\theta(x_{t-1} | x_t),$$

which is typically parameterized as a Gaussian distribution whose mean is predicted by a neural network (often a U-Net architecture). Instead of directly modeling the data distribution, the network is trained to predict the noise added at each step.

2.3 Training objective

The training objective is derived from a variational lower bound on the negative log-likelihood. In practice, this reduces to a simplified denoising objective:

$$\mathbb{E}_{x_0, \epsilon, t} \left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right],$$

where $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, and $\bar{\alpha}_t = \prod_{s=1}^t(1 - \beta_s)$. This objective encourages the model to learn how to denoise data at all noise levels.

2.4 Sampling procedure

Sampling is performed iteratively by starting from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I),$$

and applying the learned reverse transitions $p_\theta(x_{t-1} | x_t)$ sequentially until reaching x_0 . Although this process is computationally expensive due to the large number of timesteps, it yields high-quality and diverse samples because each step only requires learning a local denoising transformation.

2.5 How to choose our architecture?

We need an architecture with the following properties:

- Understand global structure
- Preserve local details
- Respond to external conditioning signals (e.g., timestep embeddings or conditional inputs)
- Remain computationally scalable since denoising is performed repeatedly across many diffusion steps

The **U-Net** architecture satisfies all of these requirements effectively.

- **Captures Both Local and Global Features**

U-Net captures both fine-grained local details and high-level global structure through its encoder–decoder architecture.

- The **encoder (downsampling path)** progressively compresses the image representation, allowing the network to learn increasingly abstract and global features.
- The **decoder (upsampling path)** reconstructs the image back to its original resolution.
- The **bottleneck** at the lowest resolution contains highly compressed global semantic information.

- **Skip connections** between corresponding encoder and decoder layers preserve spatial information and local details.

This combination enables the model to reason globally while maintaining accurate local reconstruction.

- **Same Input and Output Dimensions**

Diffusion models require predicting either:

- noise,
- denoised images,
- or velocity estimates

with the **same spatial dimensions** as the input image.

U-Net is specifically designed for dense prediction tasks where input and output resolutions must match. The decoder reconstructs feature maps back to the original image size, making it naturally compatible with diffusion objectives.

2.5.1 Inductive Biases of U-Net Relevant to Reverse Diffusion

- **Inductive Bias 1: Multi-Scale Hierarchical Representation**

U-Net’s encoder–decoder structure compresses the input into progressively smaller feature maps and then expands them back to the original resolution. This creates representations at multiple spatial scales:

- Early layers capture low-level features such as edges and textures.
- Deeper layers capture high-level semantic structure.

- * **Why This Matters for Diffusion**

Noise corruption in diffusion models is inherently scale-dependent:

- At high noise levels (large timestep t), most fine details are destroyed, and only coarse structure remains.
- At low noise levels, the model must recover fine textures and detailed spatial information.

The multi-scale hierarchy naturally enables the model to determine:

- which spatial scales still contain useful information,
- and which details must be reconstructed at each timestep.

- **Inductive Bias 2: Skip Connections Preserve Spatial Identity**

The defining feature of U-Net is its long skip connections between encoder and decoder layers at matching resolutions.

These skip connections allow the decoder to directly access spatial feature maps from the encoder instead of relying solely on the bottleneck representation.

- * **Why This Matters for Diffusion**

The noisy input x_t is not random noise; it still contains partial information about the original image x_0 , especially at lower noise levels.

Skip connections allow the network to:

- preserve spatial alignment,
- reuse existing image structure,
- and perform residual-style corrections rather than reconstructing everything from scratch.

This aligns naturally with the diffusion objective, which is fundamentally a **denoising and correction process** rather than unconditional generation.

Inductive Bias 3: Locality with Long-Range Context

Convolutions are inherently local operations. However, repeated downsampling increases the receptive field of deeper layers, allowing bottleneck features to capture global context.

At the same time, skip connections reintroduce local high-resolution information during decoding.

*** Why This Matters for Diffusion**

Effective denoising requires both:

- **Local evidence** — nearby textures, edges, and pixel relationships,
- **Global context** — object identity, shape consistency, and scene structure.

At high noise levels, global context is essential for determining plausible structure.

At low noise levels, local precision becomes critical for recovering texture and detail.

U-Net provides both simultaneously at every decoder stage through the interaction between:

- globally informed bottleneck features,
- and locally preserved encoder features.

• Inductive Bias 4: Bottleneck as a Compressed Prior

The bottleneck acts as a compressed latent representation of the image, enforcing an implicit structural prior.

Because information must pass through a low-resolution representation, the network is encouraged to retain only globally coherent structure.

*** Why This Matters for Diffusion**

At high noise levels, the model should avoid overcommitting to fine details. Instead, it should first predict:

- smooth,
- globally consistent,
- structurally plausible content.

Fine details can then be progressively refined during later denoising steps.

The bottleneck naturally supports this behavior because:

- high-frequency noise cannot easily propagate through the compressed representation,
- while global semantic structure can.

This mirrors the reverse diffusion process itself, where generation proceeds from coarse structure to fine detail over multiple timesteps.

2.6 Conditioning

In lecture 4 of the Stanford University CME 296 course, the instructor mentioned that there are three main methods for incorporating conditioning into diffusion models:

Method 1: U-Net: adding condition information

Method 2: Feature Map Modulation The feature maps are modulated through scaling and shifting operations.

Method 3: Cross-Attention Conditioning between the condition information and the feature maps.

In this experiment, I used the first method. I had already implemented AdaGN in the VAE, and adding it again would introduce additional computational overhead, while DDPMs are already relatively slow.

I also experimented with AdaGN in one of the public repositories and found that it did not provide noticeable visual improvements, nor significant quantitative gains. The notebook includes the AdaGN implementation code, which can be uncommented and tested if needed.

2.7 How does Conditioning affect?

The intuition behind why conditional DDPMs produce sharper and more visually appealing images is that an unconditional model may sample from regions that are somewhat far from the class mean Figure 1-1a. In contrast, in conditional DDPMs, the classifier provides a confidence score, and the objective is to maximize this confidence. As a result, the diffusion process is guided toward regions that are deeper within the class distribution—closer to the mean—where the generated images tend to be sharper and more coherent as image Figure 1-1b show.

3 Classifier Free Guidance CFG

3.1 Background: What Problem Are We Solving?

A standard DDPM learns to generate images by reversing a noising process. It learns a noise-prediction network $\epsilon_\theta(\mathbf{x}_t, t)$ that estimates the noise added at timestep t , given a noisy image $\mathbf{x}_t$. During training, the model sees $(\mathbf{x}_t, t)$ pairs and learns to predict the original noise.

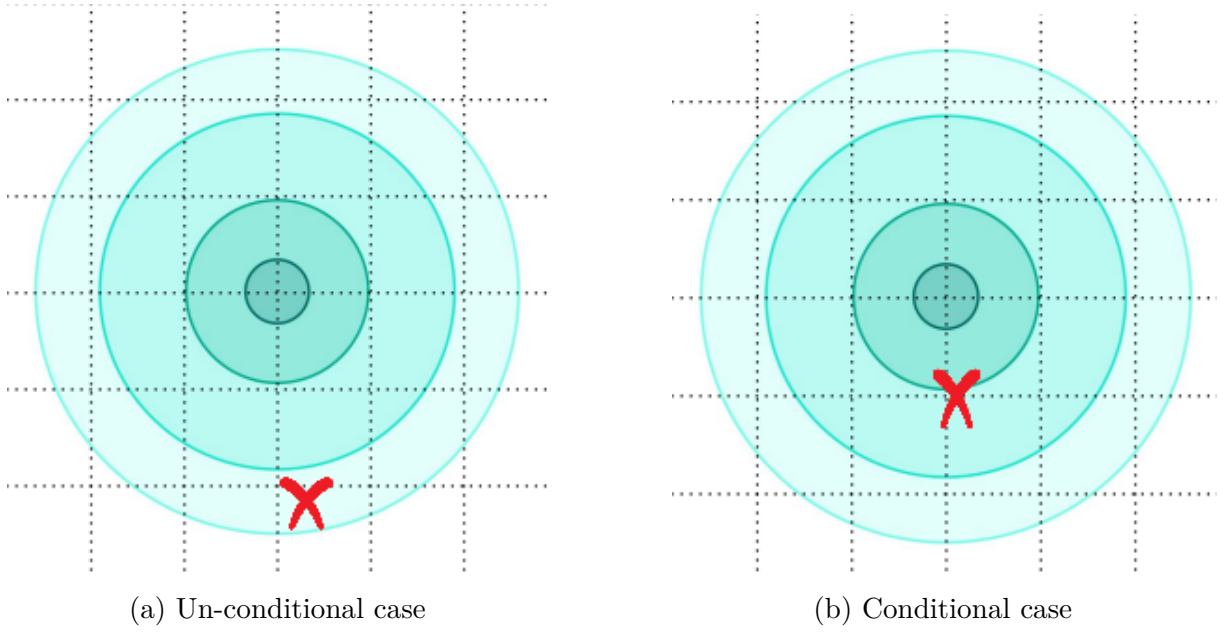


Figure 1: Comparison between the unconditional and conditional DDPMs

The core issue: **this model generates images unconditionally.** It has no mechanism to say “generate a cat” or “generate a dog.” We need a way to steer, or *guide*, the denoising process toward a desired class y .

Both CG and CFG are solutions to this steering problem, but they approach it very differently.

3.2 Part 1: Classifier Guidance (CG)

- **Step 1 — The Mathematical Foundation**

In diffusion models, the reverse process samples from:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

To condition on a class y , we want to sample from:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, y)$$

Using Bayes’ rule, we can write:

$$p(\mathbf{x}_t \mid y) \propto p(\mathbf{x}_t) \cdot p(y \mid \mathbf{x}_t)$$

Taking the log and then the gradient with respect to $\mathbf{x}_t$:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t \mid y) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(y \mid \mathbf{x}_t)$$

This means: **the score of the class-conditioned distribution = the unconditional score + the gradient from a classifier.**

- **Step 2 — What the Noise Predictor Represents**

In DDPM, the noise predictor $\varepsilon_\theta(\mathbf{x}_t, t)$ is directly related to the score function (gradient of the log probability). Specifically:

$$\varepsilon_\theta(\mathbf{x}_t, t) \approx -\sqrt{1 - \bar{\alpha}_t} \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$$

So the unconditional denoising direction is already encoded in the noise prediction network.

- **Step 3 — Introducing the Classifier**

You train a **separate classifier** $p_\phi(y | \mathbf{x}_t)$ that takes a **noisy image** $\mathbf{x}_t$ at timestep t as input and predicts the class label y . This is crucial — it must be trained on noisy images at every noise level, not clean images, because during denoising we are working with noisy $\mathbf{x}_t$.

- **Step 4 — Modifying the Noise Prediction**

Using the mathematical decomposition from Step 1, the guided noise prediction becomes:

$$\tilde{\varepsilon} = \varepsilon_\theta(\mathbf{x}_t, t) - \sqrt{1 - \bar{\alpha}_t} \cdot \gamma \cdot \nabla_{\mathbf{x}_t} \log p_\phi(y | \mathbf{x}_t)$$

where:

- $\varepsilon_\theta(\mathbf{x}_t, t)$ is the original unconditional noise prediction,
- $\nabla_{\mathbf{x}_t} \log p_\phi(y | \mathbf{x}_t)$ is the gradient of the classifier’s log-probability with respect to the noisy image,
- γ is the **guidance scale** — a scalar that controls how strongly you push toward class y .

A higher γ means the model is pushed harder toward the target class, producing more class-faithful but potentially less diverse outputs.

- **Step 5 — The Denoising Step with Classifier Guidance**

At every single reverse timestep t (from T down to 0), you:

1. Run the unconditional DDPM to get $\varepsilon_\theta(\mathbf{x}_t, t)$.
2. Feed the current noisy image $\mathbf{x}_t$ into the external classifier and compute $\nabla_{\mathbf{x}_t} \log p_\phi(y | \mathbf{x}_t)$ via backpropagation through the classifier.
3. Combine them as shown above to get the guided noise prediction $\tilde{\varepsilon}$.
4. Use $\tilde{\varepsilon}$ in place of the original ε to compute $\mathbf{x}_{t-1}$.

- **Step 6 — Class Embedding in CG?**

In pure Classifier Guidance, the **denoising network** $\varepsilon_\theta(\mathbf{x}_t, t)$ **itself is unconditional** — it does *not* take the class label y as input. The class information comes entirely from the external classifier gradient. Therefore, there is no class embedding inside the diffusion model itself in the CG setting. The diffusion model remains class-agnostic; only the classifier knows about classes.

- **Step 7 — Limitations of CG**

- You need to train a **separate classifier on noisy images** — this is non-trivial and computationally expensive.
- At every denoising step, you need to **backpropagate through the classifier** to get gradients — this slows down inference significantly.
- The classifier must generalise across all noise levels, which is a harder problem than classifying clean images.
- Any errors or biases in the classifier directly affect image generation.

3.3 Part 2: Classifier-Free Guidance (CFG)

- **Step 1 — The Core Insight**

CFG, introduced by Jonathan Ho and Tim Salimans (2022), asks: **what if we avoid training a classifier altogether?** Instead, we implicitly learn both a conditional and an unconditional noise predictor within a single network, and use the difference between them as our guidance signal.

The key insight is algebraic. Recall from Bayes’ rule:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | y) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(y | \mathbf{x}_t)$$

Rearranging:

$$\nabla_{\mathbf{x}_t} \log p(y | \mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | y) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$$

So the **implicit classifier gradient** is the difference between the conditional score and the unconditional score. If we learn both scores inside a single model, we never need an external classifier.

- **Step 2 — Training a Single Joint Model**

You train one noise prediction network $\epsilon_\theta(\mathbf{x}_t, t, y)$ that takes:

- $\mathbf{x}_t$: the noisy image at timestep t ,
- t : the timestep,
- y : the class label (or a null token $\emptyset$ representing “no class”).

During training, for each sample:

- With probability p (e.g. 10–20%), you **drop the class label** and replace y with a special null token $\emptyset$. This forces the model to learn unconditional generation.
- With probability $(1 - p)$, you **keep the class label** and train the model on conditional generation.

This single training procedure teaches the same network both:

- $\epsilon_\theta(\mathbf{x}_t, t, y)$ — the conditional noise predictor,
- $\epsilon_\theta(\mathbf{x}_t, t, \emptyset)$ — the unconditional noise predictor (via the dropout trick).

- **Step 3 — Class Embedding in CFG**

This is where class embedding becomes central. The class label y must be fed into the network in a meaningful way. Here is exactly how it works.

- **3a. Embedding the Class Label**

The discrete class label y (e.g. class index 3 for “cat”) is passed through a learned embedding table to produce a class embedding vector $\mathbf{e}_y$ of dimension d (e.g. 512-dimensional). This is analogous to token embeddings in transformers.

- **3b. Embedding the Null Token**

The null label $\emptyset$ also has its own learned embedding $\mathbf{e}_\emptyset$, which is just another row in the same embedding table. It is a trainable vector that represents “unconditional.”

- **3c. Injecting Class Conditioning into the UNet**

In DDPM, the denoising network is typically a UNet. The class embedding is injected at the same point as the timestep embedding. The timestep t is usually embedded using sinusoidal positional encoding, then passed through a small MLP to produce a timestep vector $\mathbf{e}_t$.

The class embedding $\mathbf{e}_y$ and $\mathbf{e}_t$ are **added together** (or concatenated, depending on the implementation):

$$\mathbf{c} = \mathbf{e}_t + \mathbf{e}_y$$

This combined conditioning vector $\mathbf{c}$ is then injected into each residual block of the UNet via **adaptive group normalisation (AdaGN)**, which modulates the scale and shift of feature maps at each layer:

$$\text{AdaGN}(\mathbf{h}, \mathbf{c}) = \mathbf{c}_{\text{scale}} \cdot \text{GroupNorm}(\mathbf{h}) + \mathbf{c}_{\text{shift}}$$

where $\mathbf{h}$ is the feature map, and $\mathbf{c}_{\text{scale}}$ and $\mathbf{c}_{\text{shift}}$ are linear projections of the conditioning vector. This is how class information permeates every layer of the UNet.

- **Step 4 — The CFG Guided Noise Prediction Formula**

At inference time, you run **two forward passes** through the same network:

1. **Conditional pass:** $\varepsilon_\theta(\mathbf{x}_t, t, y)$ — using the actual class embedding.
2. **Unconditional pass:** $\varepsilon_\theta(\mathbf{x}_t, t, \emptyset)$ — using the null token embedding.

Then you compute the **guided noise prediction**:

$$\tilde{\varepsilon} = \varepsilon_\theta(\mathbf{x}_t, t, \emptyset) + \gamma \cdot \left[\varepsilon_\theta(\mathbf{x}_t, t, y) - \varepsilon_\theta(\mathbf{x}_t, t, \emptyset) \right]$$

which can be rewritten as:

$$\tilde{\varepsilon} = (1 - \gamma) \cdot \varepsilon_\theta(\mathbf{x}_t, t, \emptyset) + \gamma \cdot \varepsilon_\theta(\mathbf{x}_t, t, y)$$

where γ is the guidance scale (same role as in CG):

- When $\gamma = 0$: you get pure unconditional generation.
- When $\gamma = 1$: you get standard conditional generation with no guidance boost.
- When $\gamma > 1$: you amplify the direction from unconditional toward conditional, increasing class fidelity at the cost of diversity.
- Typical values are $\gamma \in [3, 10]$ in practice.

The difference $\epsilon_\theta(\mathbf{x}_t, t, y) - \epsilon_\theta(\mathbf{x}_t, t, \emptyset)$ acts as the implicit classifier gradient, without ever having trained a classifier.

• Step 5 — The Full Denoising Loop in CFG

At every reverse timestep t from T down to 0:

1. Take the current noisy image $\mathbf{x}_t$.
2. Run the UNet with the **class embedding** $\mathbf{e}_y$ to get ϵ_{cond} .
3. Run the UNet with the **null embedding** $\mathbf{e}_\emptyset$ to get ϵ_{uncond} .
4. Compute $\tilde{\epsilon} = \epsilon_{\text{uncond}} + \gamma (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$.
5. Use $\tilde{\epsilon}$ to compute the denoised estimate of $\mathbf{x}_{t-1}$ using the standard DDPM reverse step formula.
6. Repeat.

3.4 Part 3: Direct Comparison

Training cost: CG requires training two separate models: a diffusion model and a noisy-image classifier. CFG trains a single model with label dropout — simpler and cheaper.

Inference cost: CG requires one forward pass through the diffusion model plus one backpropagation through the classifier at every step. CFG requires two forward passes through the same diffusion model at every step — no backpropagation needed.

Class information flow: In CG, the diffusion model is unconditional; class information arrives only via external gradients. In CFG, class information is baked into the model as a learned class embedding, fed through AdaGN at every residual block.

Flexibility: CFG naturally extends to text conditioning by replacing the class embedding with a text encoder (e.g. CLIP), which is exactly what Stable Diffusion does. CG is harder to extend to open-ended text because you would need a classifier over all possible text prompts.

Quality vs. diversity tradeoff: Both methods use the scalar γ to navigate this tradeoff. CFG tends to produce higher quality results in practice and has become the standard approach in modern diffusion models.

4 DDPMs in Contrast to Variational Autoencoders

Denoising Diffusion Probabilistic Models (DDPMs) and Variational Autoencoders (VAEs) are both latent-variable generative models, but they differ fundamentally in how they

represent uncertainty, structure their latent space, and impose inductive bias on the generative process. Understanding these differences is important for explaining why VAEs often produce faster but blurrier samples, while DDPMs tend to generate sharper and more detailed outputs at the cost of iterative sampling.

Variational Autoencoders learn a compressed latent representation by mapping data x into a continuous latent variable z through an encoder network, and reconstructing x via a decoder. The training objective is based on maximizing the Evidence Lower Bound (ELBO), which balances reconstruction fidelity with a Kullback–Leibler divergence term that regularizes the approximate posterior $q_\phi(z|x)$ toward a simple prior $p(z)$, typically a standard Gaussian:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) \parallel p(z)).$$

The inductive bias in VAEs is therefore strongly tied to *global latent compression*: the model assumes that data can be explained by a low-dimensional, smooth, and approximately Gaussian latent space. This encourages semantic continuity and interpolation within the latent representation, but it often leads to over-regularization, where high-frequency details are suppressed, resulting in blurry reconstructions and generated samples.

In contrast, DDPMs model data generation as a gradual denoising process defined over a fixed Markov chain. Instead of learning a single global latent representation, DDPMs define a forward diffusion process that incrementally corrupts data with Gaussian noise:

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right),$$

where β_t controls the variance schedule. The model then learns a reverse process that iteratively removes noise:

$$p_\theta(x_{t-1} | x_t).$$

The inductive bias in DDPMs is fundamentally *local and incremental*. Rather than compressing all information into a bottleneck latent variable, the model assumes that complex data distributions can be transformed into Gaussian noise through a sequence of small, approximately reversible perturbations. Each denoising step corresponds to solving a simpler local reconstruction problem, which enables the model to preserve fine-grained structure and generate highly realistic samples.

A useful way to contrast the two approaches is through their treatment of information bottlenecks. VAEs impose a *global bottleneck* through the latent variable z , forcing all information about the input to pass through a compact representation. This can encourage disentangled and interpretable latent factors, but may discard subtle image details. DDPMs, by contrast, avoid a single compression bottleneck and instead distribute the generative process across many intermediate noise levels. In this sense, DDPMs replace latent compression with a hierarchy of progressively refined denoising tasks.

This difference in inductive bias—global latent compression in VAEs versus local denoising consistency in DDPMs—largely explains the empirical distinction between the two models. VAEs provide efficient inference and fast sampling, whereas DDPMs achieve superior perceptual quality and diversity through iterative refinement.

Here is a question that might come to mind, **Why we didn’t add skip connections to the VAE?**

As in the VAE, we generate the images in one shot, the presence of skip connections is kind of cheating, since it weakens the latent bottleneck. Also, during inference/generation, there is no encoder available.

But in diffusion models, we convert the generation process into multiple steps of noise prediction, and that is why we add skip connections. In this case, what we encode is not directly what we need to generate, so the skip connections do not really act as cheating.

5 Experimental Setup

I Implement the conditional version as Conditioning helps achieve good results with fewer epochs and diffusion steps. As observed in the VAE experiments, it consistently produces higher-quality images.

It improves convergence speed. It generates better visual results early in training. It allows easier control over the generated outputs.

5.1 Dataset

All experiments use **CIFAR-10** (Canadian Institute For Advanced Research) which is a foundational dataset in computer vision used to train and evaluate image generation models. It consists of 60,000 32×32 pixel color images across 10 distinct, mutually exclusive object and animal classes.

5.2 Evaluation Metrics

FID Fréchet Inception Distance — measures the distributional gap between real and generated images in Inception feature space. *Lower is better.*

IS (Inception Score) Evaluates both the quality and diversity of generated images. Reported as mean $\pm$ standard deviation over multiple splits. *Higher is better.*

6 Experiment 1

6.1 Motivation

As we see in the VAE experiments, the VAE suffers from blurriness. Therefore, we now explore DDPMs to investigate whether they can produce better results.

6.2 Hyperparameters

you can find it in the DDPM Notebook.

6.3 Evaluation Results

Table 1: Experiment 1— Quantitative evaluation (FID and IS). FID: lower is better; IS: higher is better.

FID ↓	IS mean ↑
17.2369	8.6970

6.4 Qualitative Results

6.4.1 Random Samples

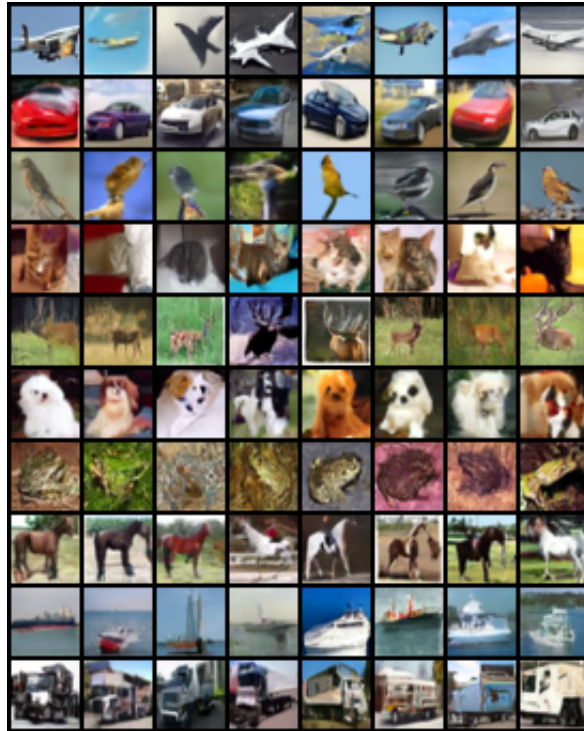


Figure 2: Experiment 1— Each row correspond to class

6.5 Analysis and Observations

- **Images quality** The images have good quality and coherence, and they appear realistic.
- **Diversity** As we can see in each row, the model has the ability to generate diverse images under different conditions, such as varying poses and backgrounds.
- **Achieving Better Results from the best VAE model** DDPMs achieve higher Inception Scores and lower FID scores than VAEs, demonstrating superior generative performance.

7 Conclusions

In summary, DDPMs outperform VAEs in generating high-fidelity images. The progressive denoising mechanism employed by DDPMs enables the model to iteratively refine image details, leading to sharper and more realistic outputs. In contrast, VAEs commonly suffer from blurry reconstructions due to the probabilistic nature of latent variable approximation. Therefore, DDPMs provide superior visual quality and better detail preservation in image generation tasks.